Tools and execution limits

INSTRUCTOR ANSWERS - STAFF ONLY

Core questions 1-5

Use groups of five at suitable points in the session. Ask learners to explain their choice and one rejected option. Use the final five if helpful. Keep these group discussions separate from the existing individual knowledge-check scores.

1. A developer sends the lookup_case tool to the model in the request's toolConfig. What does the model learn about the tool?

Correct answer: C. It learns the tool's name, its description and the shape of its input, but never the function's code.

A tool definition gives the model a name, a description and an input schema, and the model uses them to decide when to ask for the tool and what to send. The model never sees the function's code and can't call it. The caller scope stays in application state, where the dispatcher reads it.

2. Each option describes a Kestrel task. Which task is better handled by ordinary application code than by a model with tools?

Correct answer: C. Every customer whose case has waited 48 hours for their reply receives a reminder from Kestrel.

A rule with fixed inputs, such as a reminder after 48 hours, needs the same function call every time, so code can make it without a model's latency or cost. The other tasks start from varied wording or need an explanation in words, which suits a model that can choose the right tool. AWS's term reactive agent also covers fixed triggers like this one.

3. The description of draft_reply says only "Creates a reply for a case." What is the most likely consequence?

Correct answer: A. The model may tell the user that the reply was sent, as nothing says that a supervisor approves it.

A description must say what the tool doesn't do when a reader might assume it does. Without a statement that a supervisor approves every draft and that nothing is sent, the model may report a reply as sent although the tool only stored a draft. The dispatcher checks names, inputs and scope, not the description, and the API doesn't require a description at all.

Tools and execution limits

INSTRUCTOR ANSWERS - STAFF ONLY

4. A developer sets the strict flag on lookup_case and proposes removing the application's own input validation. Why does the course keep it?

Correct answer: B. The application validates every tool request before execution, including requests from outside the model.

The strict flag asks Bedrock to constrain the model's tool request to the schema, but the application still validates every request before anything runs. That check protects the function from invalid arguments whatever their source, including constructed requests in tests. A number such as 1036, a list or an extra field fails validation, and the tool doesn't run.

5. Day 8 serves one of the assistant's tools from an MCP server. Why does the exam guide speak of standard function definitions for this kind of tool?

Correct answer: D. The name, description and input schema carry over almost unchanged from Converse to MCP.

The MCP specification describes a tool with a name, a description and an inputSchema in JSON Schema, the same three parts that a Converse toolSpec uses. The same definition therefore serves the model whichever way the tool is reached. MCP adds optional fields such as outputSchema and annotations, but it keeps the input schema.

Discussion: Ask the group to connect one of these ideas to today's Kestrel example. Use the Day 3 learner handbook to resolve uncertainty.

Tools and execution limits

INSTRUCTOR ANSWERS - STAFF ONLY

Core questions 6-10

6. A learner sets toolChoice to the named form for lookup_case and sends the request to a model that doesn't support it. What happens?

Correct answer: C. The request fails with a ValidationException, so the learner should test the model first.

Support for any and the named form depends on the individual model, and an unsupported setting fails the request with a ValidationException instead of falling back to auto. Bedrock never runs a Converse tool itself, because tool use is client side. Support Assistant v1 sends auto, and nothing in its loop depends on forcing a tool.

7. A Converse response ends with the stop reason tool_use and contains a toolUse block for lookup_case. What must the application do with the block's toolUseId?

Correct answer: A. Copy it exactly into the matching toolResult, without parsing it, because its format can vary.

The toolResult must repeat the toolUseId so that the model can match the result to its request. Bedrock generates the identifier for tools you define, and some models pass their provider's identifier through, so its format varies and code shouldn't parse it. The model matches by identifier, not by position, which is why dropping the identifier would fail.

8. A run about case_1017 reaches its third model call. What must that request contain?

Correct answer: D. It must contain every message in order, with each toolUse and toolResult, and the same toolConfig.

Converse is stateless: Bedrock doesn't store the conversation, so each request carries the whole history, including every tool request and result. The same toolConfig goes with every request, because the model needs the definitions each time it reads the history. The input tokens, and so the cost, grow with each call.

Tools and execution limits

INSTRUCTOR ANSWERS - STAFF ONLY

9. A developer's lookup_case returns the text "Error: case not found" with the status success when a case is missing. Why is this a problem?

Correct answer: D. The model may treat the error text as case data, because the status tells it the call worked.

A failure must be returned with the status error and a structured message that names the failed check and suggests a permitted next step. With the status success, the model reads the text as the tool's real output and may build an answer on it. Catch only the exceptions you expect, and report them as errors.

10. The model asks for lookup_case with the input {"case_id": 1036}, a number, in a run scoped to case_1035. Which rule does the dispatcher's log line name?

Correct answer: C. It names input_schema, because a number fails the pattern and input is checked before scope.

The dispatcher checks the allowlist, the time left, the input, repeated actions and then the scope, and it stops at the first check that fails. lookup_case is on the allowlist, but a number doesn't match the string pattern, so the input_schema rule refuses the request. The input check comes before the scope check so that the scope check reads a validated case_id.

Discussion: Ask the group to connect one of these ideas to today's Kestrel example. Use the Day 3 learner handbook to resolve uncertainty.

Tools and execution limits

INSTRUCTOR ANSWERS - STAFF ONLY

Core questions 11-15

11. A run's deadline has passed when the model asks for issue_refund. Which rule does the dispatcher's log line name?

Correct answer: A. It names allowlist, because the allowlist is checked before the time left.

The allowlist comes first, so an unknown tool name never reaches any other code, and the dispatcher stops at the first check that fails. issue_refund isn't on the allowlist, so the line names allowlist even though the deadline has also passed. A request for an allowed tool after the deadline would meet time_limit.

12. Another design would silently replace the model's case_id with the case that the application set. What does the course's design gain by refusing the mismatch instead?

Correct answer: B. It records every attempt to reach another case as a refusal, so the attempts can be found and counted.

Replacing the identifier would keep the lookup on the intended case, but it wouldn't record that the model asked for another one. Refusing with the scope rule writes a log line for every such attempt. The scope comes from application state, never from the model's response or the user's message, so no confirmation can widen it.

13. A first-line agent may read every technical support case. A run that drafts a reply on case_1026 is given the agent's whole team scope. Why is this a fault?

Correct answer: D. The scope doesn't fit the task, so a lookup of another case passes and a draft could quote it.

The checks work exactly as written, but a scope wider than the task lets a lookup of case_1002 pass. A draft that followed the note could then quote another customer's case to the customer of case_1026. The fix is to scope each run to the one case that the agent opened, which is least privilege applied to a run.

Tools and execution limits

INSTRUCTOR ANSWERS - STAFF ONLY

14. MAX_MODEL_CALLS in config.py is 6. Why does the loop count model calls rather than tool calls?

Correct answer: B. A model call is the unit that costs money, takes time and appears in the ledger as one attempt.

The iteration limit is the loop's stopping condition, and it counts the unit that costs money and time and that the ledger records. A lookup followed by an answer uses two calls, and so does a lookup followed by a draft, so six leaves room for the model to correct a refused request. Bedrock sets no limit on model calls per run; the limit is the application's.

15. The loop checks its 180-second deadline before each model call. Which limit stops a single model call that hangs after it has started?

Correct answer: D. The Day 2 client's read timeout limits how long the client waits for that one response.

A check before each call can't interrupt a call that is already running. The client's read timeout bounds each call, and the run's deadline stops a new call from starting once the time is up. The dispatcher's time_limit rule applies to tool requests, not to model calls in progress.

Discussion: Ask the group to connect one of these ideas to today's Kestrel example. Use the Day 3 learner handbook to resolve uncertainty.

Tools and execution limits

INSTRUCTOR ANSWERS - STAFF ONLY

Core questions 16-20

16. The dispatcher refused a lookup of case_1002 with the scope rule. Later in the same run, the model sends the identical request. What does the second log line show, and where is the original reason?

Correct answer: C. It shows repeated_action, and the first refusal's line gives the original reason, scope.

The dispatcher keeps the tool name and an input hash for every request in the run, including refused ones, and the repeated-action check comes before the scope check. The second identical request therefore meets repeated_action. To find why the request was first refused, read the earlier line, which names scope.

17. Kestrel adds a guardrail with a prompt attack filter. Why must the dispatcher still check the tool requests that follow a lookup of case_1026?

Correct answer: C. The case note arrives in a toolResult block, and no guardrail filter evaluates tool results.

A guardrail's filters evaluate user messages, model responses and a system prompt marked for guarding, but not toolResult, toolSpec or toolUse.input. The injected note in case_1026 arrives in a tool result, so the filter never sees it. The dispatcher's allowlist and scope checks therefore remain the controls that refuse the note's demands.

18. case_1035 has four notes on clean records. A run ends awaiting_approval, and a supervisor then approves the draft. How many notes does the case have after the run and after the approval?

Correct answer: B. It has four after the run and five after the approval, because only approval adds the draft.

A pending draft is stored separately from the case history, so the run leaves the count at four. Only approve_draft(), called with a supervisor's decision, adds the draft to the history, marked with approved_by set to supervisor. The approval itself is recorded in a log line, not as a second note.

Tools and execution limits

INSTRUCTOR ANSWERS - STAFF ONLY

19. The assistant's answer gives the correct status of case_1017. How do you show from the records that the run actually looked the case up?

Correct answer: A. Find a lookup_case request for case_1017 and a toolResult with the same toolUseId and the status success.

A model can write a correct-looking answer without a lookup, by reusing earlier context or inventing details, so the wording proves nothing. The evidence is the toolUse block for lookup_case with that case_id and the matching toolResult with the status success. If either block is missing, the history doesn't show a successful lookup.

20. Each decision log line records input_digest, a short SHA-256 hash, instead of the request's input. Why?

Correct answer: A. A draft_reply input holds reply text that can quote a customer, and the hash keeps it out of the log.

Reply text can quote what a customer wrote, so the log stores a hash instead. Two identical inputs give the same hash, which is enough to show a repeat without storing either input. A hash can't be reversed to recover the text, and constructed test requests get digests too.

Discussion: Ask the group to connect one of these ideas to today's Kestrel example. Use the Day 3 learner handbook to resolve uncertainty.

Tools and execution limits

INSTRUCTOR ANSWERS - STAFF ONLY

Optional extra questions 21-25

21. The lookup tool will later run in an AWS Lambda function with its own timeout. Why does the loop still need a deadline for the whole run?

Correct answer: B. The Lambda timeout bounds each call, up to 900 seconds, but not the total time of a run.

A Lambda function's timeout defaults to 3 seconds and can be set up to 900, so it limits each call to the tool, like the client's read timeout. A run that makes several calls still needs a deadline for its total time. A circuit breaker stops calls to a failing service for a while; it doesn't replace either limit.

22. In a Step Functions version of the approval step, a task sends a task token to an approver and waits. What should a production design add?

Correct answer: A. A heartbeat timeout, HeartbeatSeconds, that fails the task when the token doesn't return in time.

With the callback pattern, a task in a Standard workflow can wait until the execution reaches its one-year limit, so a production approval step needs a heartbeat timeout. A model's stated confidence is generated text and never authorises an action. The model must have no path to approval, in Step Functions or in Python.

23. The instructor tests four wordings of a request for case_1002. Why does each wording run in a new run?

Correct answer: B. Within one run, a second request for case_1002 would meet repeated_action instead of scope.

The dispatcher remembers every request in a run, including refusals, so a repeat of the same lookup would be refused by the repeated-action check. A new run for each wording shows that the scope rule refuses every wording, because the dispatcher compares the validated case_id with the scope and ignores the wording. Converse is stateless, so the model carries nothing between runs.

Tools and execution limits

INSTRUCTOR ANSWERS - STAFF ONLY

24. In one run, the model declines to look up case_1002 and asks for no tool. What do the records show about this refusal?

Correct answer: D. Only the message history shows it, because a refusal that the model chose leaves no log line.

A refusal from code has a log line with a rule, but a refusal that the model chose leaves only text in the message history. Record it as a model refusal. Together, the two records show which part of the system made each decision, and a later run may behave differently, which is why the dispatcher is tested directly.

25. Under Kestrel's goodwill credit policy, a first-line agent proposes a credit of £60.00 on a case. Who must approve it?

Correct answer: C. A supervisor must, because the credit is above £20.00 and within the supervisor's £150.00.

A first-line agent can give up to £20.00, a supervisor can approve up to £150.00, and a credit above £150.00 needs a billing manager. A credit of £60.00 therefore needs a supervisor. Version 1 has no credit tool, and Week 3 uses Cedar policies to check the role and the amount before such an action.

Discussion: Ask the group to connect one of these ideas to today's Kestrel example. Use the Day 3 learner handbook to resolve uncertainty.